

Binary Information

Binary Name => crackme1

Language => C/C++

Arch => x86x64

Platform => Unix/Linux

```
$ file crackme1
```

```
crackme1: ELF 64-bit LSB executable, x86-64, version 1 (SYSV), dynamically linked,  
interpreter /lib64/ld-linux-x86-64.so.2, for GNU/Linux 2.6.24,  
BuildID[sha1]=15edebdd57e083d5f5a69544987f9983560273e0, with debug_info, not  
stripped
```

Analysis

Static Analysis

- The program takes username as input which has to be between 8 and 12 characters otherwise the message **username must be between 8 and 12!** is printed.
- After taking the username the program takes a serial number as input

Below is a portion of the **main()** disassembly of the program you need to be concerned about.

```
.....(SNIP).....  
0x0000000000401392 <+773>: ucomisd xmm0,xmm1  
0x0000000000401396 <+777>: jne 0x4013b6 <main+809>  
0x0000000000401398 <+779>: mov esi,0x4015ea  
0x000000000040139d <+784>: mov edi,0x602240  
0x00000000004013a2 <+789>: call 0x400eb0  
<_ZStlsISt11char_traitsIcEERSt13basic_ostreamIcT_ES5_PKc@plt>  
0x00000000004013a7 <+794>: mov esi,0x400f50  
0x00000000004013ac <+799>: mov rdi,rax  
0x00000000004013af <+802>: call 0x400f30 <_ZNSolsEPFRSoS_E@plt>  
0x00000000004013b4 <+807>: jmp 0x4013d2 <main+837>  
0x00000000004013b6 <+809>: mov esi,0x4015f2  
0x00000000004013bb <+814>: mov edi,0x602240  
0x00000000004013c0 <+819>: call 0x400eb0  
<_ZStlsISt11char_traitsIcEERSt13basic_ostreamIcT_ES5_PKc@plt>  
.....(SNIP).....  
gef> x/s 0x4015f2  
0x4015f2: "s/n WRONG!"  
gef> x/s 0x4015ea  
0x4015ea: "s/n OK!"  
gef>
```

- Here the SSE registers(**xmm0**, **xmm1**) are compared. If they are equal then print **Ok!** else print **WRONG !**.

Dynamic Analysis

Prediction 1

- Setup a breakpoint at **main()** and at 0x0000000000401392 <+773>: `ucomisd xmm0,xmm1`.
- Inputs given => username => 12345678 serial => 1234
- Then I ran the program and analysed the values of **xmm0** and **xmm1** registers at the 2nd breakpoint.

```
gef> p $xmm0
$5 = {
  v8_bfloat16 = {0, 0, -1.29e-22, 16.88, 0, 0, 0, 0},
  v8_half = {0, 0, -0.0034714, 2.7637, 0, 0, 0, 0},
  v4_float = {0, 16.950737, 0, 0},
  v2_double = {49505152, 0},
  v16_int8 = {0x0, 0x0, 0x0, 0x0, 0x1c, 0x9b, 0x87, 0x41, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0},
  v8_int16 = {0x0, 0x0, 0x9b1c, 0x4187, 0x0, 0x0, 0x0, 0x0},
  v4_int32 = {0x0, 0x41879b1c, 0x0, 0x0},
  v2_int64 = {0x41879b1c00000000, 0x0},
  uint128 = 0x41879b1c00000000
}
gef> p $xmm1
$6 = {
  v8_bfloat16 = {0, 0, 1.311e+05, 4.594, 0, 0, 0, 0},
  v8_half = {0, 0, 8, 2.2871, 0, 0, 0, 0},
  v4_float = {0, 4.60253906, 0, 0},
  v2_double = {1234, 0},
  v16_int8 = {0x0, 0x0, 0x0, 0x0, 0x0, 0x48, 0x93, 0x40, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0},
  v8_int16 = {0x0, 0x0, 0x4800, 0x4093, 0x0, 0x0, 0x0, 0x0},
  v4_int32 = {0x0, 0x40934800, 0x0, 0x0},
  v2_int64 = {0x4093480000000000, 0x0},
  uint128 = 0x4093480000000000
}
gef>
```

- Notice something here —> **xmm1 = 1234** (user input serial number) while **xmm0 = 49505152** which seems like ASCII values char 1 = 49 , 2 = 50, 3 = 51, 4 = 52. So I figured to give serial numbers corresponding to the ascii values as input.

Prediction 2

- username = ABCDEFGH and serial => 979899100
- Checking the **xmm0** and **xmm1** registers again we find....

```
gef> p $xmm0
$9 = {
  v8_bfloat16 = {0, 2, 8.397e+05, 18.88, 0, 0, 0, 0},
  v8_half = {0, 2, 10.602, 2.7949, 0, 0, 0, 0},
  v4_float = {2, 18.9107914, 0, 0},
  v2_double = {97669968, 0},
  v16_int8 = {0x0, 0x0, 0x0, 0x40, 0x4d, 0x49, 0x97, 0x41, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0},
  v8_int16 = {0x0, 0x4000, 0x494d, 0x4197, 0x0, 0x0, 0x0, 0x0},
  v4_int32 = {0x40000000, 0x4197494d, 0x0, 0x0},
  v2_int64 = {0x4197494d40000000, 0x0},
  uint128 = 0x4197494d40000000
}
gef> p $xmm1
$10 = {
  v8_bfloat16 = {0, 9.904e+27, 1.276e-07, 25.62, 0, 0, 0, 0},
  v8_half = {0, 6144, 0.2522, 2.9004, 0, 0, 0, 0},
  v4_float = {9.90352031e+27, 25.6504078, 0, 0},
  v2_double = {979899100, 0},
  v16_int8 = {0x0, 0x0, 0x0, 0x6e, 0x9, 0x34, 0xcd, 0x41, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0},
  v8_int16 = {0x0, 0x6e00, 0x3409, 0x41cd, 0x0, 0x0, 0x0, 0x0},
  v4_int32 = {0x6e000000, 0x41cd3409, 0x0, 0x0},
  v2_int64 = {0x41cd34096e000000, 0x0},
  uint128 = 0x41cd34096e000000
}
gef>
```

Here A = 97 (predicted), C = 99 (predicted) but the rest of the values are different.

- I understood later on that the odd positions are converted to the uppercase and even positions of characters to lowercase. The values are different because the uppercase and lowercase of the characters have different values. You can cross verify from the **ascii table**.
- If you have noticed whatever values you input the total length of the serial number is always 8.

Testing 1

- Now lets supply usernames greater than and equal to 8 characters but less than 12 characters.

```
username1 = "abcdefgh" # 8 characters
username2 = "abcdefghi" # 9 characters
username3 = "abcdefghij" # 10 characters
username4 = "abcdefghijk" # 11 characters
username5 = "abcdefghijkl" # 12 characters
```

After checking all the values we get the below corresponding keys of each individual username.

```
username1 = "abcdefgh" <-> key = 97669968
username2 = "abcdefghi" <-> key = 66996810
username3 = "abcdefghij" <-> key = 99681017
username4 = "abcdefghijk" <-> key = 68101701
username5 = "abcdefghijkl" <-> key = 10170103
```

- The length of the serial number is 8 but...
 1. bytes (0-7) => 8 characters username
 2. bytes (2-9) => 9 characters username
 3. bytes (4-11) => 10 characters username
 4. bytes (6-13) => 11 characters username
 5. bytes (8-15) => 12 characters username

So based on this I formed a keygen program to generate the possible keys.

[KeyGen.py](#)

Click the link =>

<https://gist.github.com/SHROUDSOURAV/279e1aa1edb6ae01a3e85edb61e856bf>

The keys generated by the program

OUTPUT

```
username => abcdefgh and key => 97669968
username => abcdefghi and key => 66996810
username => abcdefghij and key => 99681017
username => abcdefghijk and key => 68101701
username => abcdefghijkl and key => 10170103
```

Testing our keys

```
└─$ ./crackme1
username:
abcdefgh
serial number:
97669968
s/n OK!
```

```
└─$ ./crackme1
username:
abcdefghi
serial number:
66996810
s/n OK!
```

```
└─$ ./crackme1
username:
abcdefghij
serial number:
99681017
```

```
└─$ ./crackme1
username:
abcdefghijk
serial number:
68101701
s/n OK!
```

```
└─$ ./crackme1
username:
abcdefghijkl
serial number:
10170103
s/n OK!
```

Finally, we can see that our **KeyGen.py** successfully generated the right combination of keys.